

EXPERIMENT 20

By Trishil Chauhan

Aim

To design algorithms and test cases and develop a menu-driven modular C project for geometrical measurements and physical properties of sphere, cylinder, cone and cuboid using only fundamental data types, call-by-value functions, selection statements and loops.

Given Requirements

The project provides surface area, volume, base area, mass, weight, gravitational potential energy, pressure, buoyant force and buoyancy behavior. $\text{PI} = 3.1416$, $\text{GRAVITY} = 9.8$ and $\text{TOLERANCE} = 0.000001$. Cone slant height is obtained with Newton-Raphson square root rather than math.h .

Algorithms

1. Main menu

Repeat: display options 1-10; read and validate the choice; use switch-case to call the required operation; stop only when option 10 is selected.

2. Sphere

Area = $4 * \text{PI} * r^2$. Volume = $(4/3) * \text{PI} * r^3$. Radius must be greater than zero.

3. Cylinder

Area = $2 * \text{PI} * r * (r+h)$. Volume = $\text{PI} * r^2 * h$. Base area = $\text{PI} * r^2$. Radius and height must be positive.

4. Cone

Compute slant height = $\sqrt{r^2+h^2}$ using `calculate_square_root()`. Area = $\text{PI} * r * (r+\text{slant})$. Volume = $(1/3) * \text{PI} * r^2 * h$. Base area = $\text{PI} * r^2$.

5. Cuboid

Area = $2 * (l*b + b*h + l*h)$. Volume = $l*b*h$. Base area = $l*b$. All dimensions must be positive.

6. Mass and weight

Obtain solid volume. Mass = density * volume. Weight = mass * GRAVITY. Density must be positive.

7. Potential energy

Obtain mass and validate elevation ≥ 0 . Potential energy = mass * GRAVITY * elevation.

8. Pressure

For cylinder/cone use circular base area; for cuboid use $l*b$. Pressure = $\text{density} * \text{volume} * \text{GRAVITY} / \text{contact_area}$. Sphere is rejected.

9. Buoyancy

Buoyant force = $\text{fluid_density} * \text{volume} * \text{GRAVITY}$. If solid density > fluid density: SINKS; if less: FLOATS; if equal: SUSPENDED.

10. Newton-Raphson square root

If $n < 0$ return -1; if $n = 0$ return 0. Set $\text{old} = n$ and repeatedly $\text{new} = 0.5 * (\text{old} + n/\text{old})$; take absolute difference manually; $\text{old} = \text{new}$; continue while difference $\geq \text{TOLERANCE}$; return new.

Test-Case Table

TC	Operation / Input	Type	Expected result
1	Sphere $r=1$	Valid	Area=12.5664 m ² ; Volume=4.1888 m ³
2	Cylinder $r=1, h=2$	Valid	Area=18.8496 m ² ; Volume=6.2832 m ³
3	Cone $r=3, h=4$	Valid / known sqrt	Slant=5; Area=75.3984 m ² ; Volume=37.6992 m ³

TC	Operation / Input	Type	Expected result
4	Cuboid l=2,b=3,h=4	Valid	Area=52 m ² ; Volume=24 m ³ ; Base=6 m ²
5	Radius=0 then 2	Invalid / boundary	Reject 0 and request radius again
6	Height=-1 then 1	Invalid	Reject negative height and request again
7	Base Area -> Sphere	Invalid selection	Display that sphere base area is not defined; return to menu
8	Elevation=0	Boundary	Accept; potential energy = 0 J
9	Density=0 then 1000	Invalid / boundary	Reject 0; accept positive density
10	Solid density 800, fluid 1000	Valid	FLOATS
11	Solid density 1000, fluid 1000	Boundary	SUSPENDED
12	Solid density 1200, fluid 1000	Valid	SINKS
13	sqrt(0)	Boundary	Returns 0
14	sqrt(-4)	Invalid for real sqrt	Returns -1
15	Menu choice 11 then 10	Invalid	Reject 11; accept Exit=10

Modular C Project

solid.h

```
#ifndef SOLID_H
#define SOLID_H
#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001
#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4
#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

double calculate_square_root(double number);
double sphere_area(double radius);
double sphere_volume(double radius);
double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);
double cone_area(double radius, double height);
double cone_volume(double radius, double height);
double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);
double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);
double calculate_mass(double density);
double calculate_weight(void);
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density, double fluid_density);
#endif
```

math_utils.c

```
#include "solid.h"
double calculate_square_root(double number) {
    double old_value, new_value, difference;
    if (number < 0) return -1;
    if (number == 0) return 0;
    old_value = number;
    do {
        new_value = 0.5 * (old_value + number / old_value);
        difference = new_value - old_value;
        if (difference < 0) difference = -difference;
        old_value = new_value;
    } while (difference >= TOLERANCE);
    return new_value;
}
```

geometry.c

```
#include "solid.h"
double sphere_area(double r){ return 4*PI*r*r; }
double sphere_volume(double r){ return (4.0/3.0)*PI*r*r*r; }
double cylinder_area(double r,double h){ return 2*PI*r*(r+h); }
double cylinder_volume(double r,double h){ return PI*r*r*h; }
double cone_area(double r,double h){
    double s=calculate_square_root(r*r+h*h);
    return PI*r*(r+s);
}
double cone_volume(double r,double h){ return (1.0/3.0)*PI*r*r*h; }
double cuboid_area(double l,double b,double h){
    return 2*(l*b+b*h+l*h);
}
double cuboid_volume(double l,double b,double h){ return l*b*h; }
double circular_base_area(double r){ return PI*r*r; }
double cuboid_base_area(double l,double b){ return l*b; }
```

physics.c

```
#include <stdio.h>
#include "solid.h"

static double positive(const char *msg){
    double x; do { printf("%s",msg); scanf("%lf",&x);
    if(x<=0) printf("Invalid value. Enter a positive value.\n"); } while(x<=0);
    return x;
}

static int solid_choice(int allowSphere){
    int s; do { printf("Solid: 1 Sphere, 2 Cylinder, 3 Cone, 4 Cuboid: "); scanf("%d",&s);
    if(s<1||s>4||(!allowSphere&&s==SPHERE)) printf("Invalid/unsupported solid. Try again.\n");
    } while(s<1||s>4||(!allowSphere&&s==SPHERE)); return s;
}

static double get_volume(int s){
    double r,h,l,b;
    if(s==SPHERE){r=positive("Radius (m): "); return sphere_volume(r);}
    if(s==CYLINDER){r=positive("Radius (m): ");h=positive("Height (m): ");return cylinder_volume(r,h);}
    if(s==CONE){r=positive("Radius (m): ");h=positive("Height (m): ");return cone_volume(r,h);}
}
```

```

        l=positive("Length (m): ");b=positive("Breadth (m): ");h=positive("Height (m): ");return cuboid_volume(l,b,h);
    }
    double calculate_mass(double density){ return density*get_volume(solid_choice(1)); }
    double calculate_weight(void){ double d=positive("Density (kg/m^3): "); return calculate_mass(d)*GRAVITY; }
    double potential_energy(double elevation){ double d=positive("Density (kg/m^3): "); return calculate_mass(d)*GRAVITY*elevation; }
    double calculate_pressure(double density){
        int s=solid_choice(0); double r,h,l,b,v,a;
        if(s==CYLINDER){r=positive("Radius (m): ");h=positive("Height (m): ");v=cylinder_volume(r,h);a=circular_base_area(r);}
        else if(s==CONE){r=positive("Radius (m): ");h=positive("Height (m): ");v=cone_volume(r,h);a=circular_base_area(r);}
        else {l=positive("Length (m): ");b=positive("Breadth (m): ");h=positive("Height (m): ");v=cuboid_volume(l,b,h);a=cuboid_base_area(l,b,h);}
        return density*v*GRAVITY/a;
    }
    double buoyant_force(double fluid_density){ return fluid_density*get_volume(solid_choice(1))*GRAVITY; }
    int determine_buoyancy(double sd,double fd){ if(sd>fd)return SINKS; if(sd<fd)return FLOATS; return SUSPENDED; }

```

main.c

```

#include <stdio.h>
#include "solid.h"
static double pos(const char *m){double x;do{printf("%s",m);scanf("%lf",&x);if(x<=0)printf("Invalid. Try again.\n");}while(x<=0);return x;}
static int solid(void){int s;do{printf("1 Sphere  2 Cylinder  3 Cone  4 Cuboid: ");scanf("%d",&s);if(s<1||s>4)printf("Invalid solid.\n");}while(s<1||s>4);return s;}
int main(void){
    int ch,s,res; double r,h,l,b,d,e,fd,sd;
    do{
        printf("\n1 Surface Area\n2 Volume\n3 Base Area\n4 Mass\n5 Weight\n6 Potential Energy\n7 Pressure\n8 Buoyant Force\n9 Buoyancy Behavior\n10 Exit\n");
        scanf("%d",&ch); if(ch<1||ch>10){printf("Invalid menu choice.\n");continue;}
        switch(ch){
            case 1:s=solid(); if(s==SPHERE){r=pos("Radius: ");printf("Area = %.4f m^2\n",sphere_area(r));}
            else if(s==CYLINDER){r=pos("Radius: ");h=pos("Height: ");printf("Area = %.4f m^2\n",cylinder_area(r,h));}
            else if(s==CONE){r=pos("Radius: ");h=pos("Height: ");printf("Area = %.4f m^2\n",cone_area(r,h));}
            else{l=pos("Length: ");b=pos("Breadth: ");h=pos("Height: ");printf("Area = %.4f m^2\n",cuboid_area(l,b,h));} break;
            case 2:s=solid(); if(s==SPHERE){r=pos("Radius: ");printf("Volume = %.4f m^3\n",sphere_volume(r));}
            else if(s==CYLINDER){r=pos("Radius: ");h=pos("Height: ");printf("Volume = %.4f m^3\n",cylinder_volume(r,h));}
            else if(s==CONE){r=pos("Radius: ");h=pos("Height: ");printf("Volume = %.4f m^3\n",cone_volume(r,h));}
            else{l=pos("Length: ");b=pos("Breadth: ");h=pos("Height: ");printf("Volume = %.4f m^3\n",cuboid_volume(l,b,h));} break;
            case 3:s=solid(); if(s==SPHERE){printf("Base area of a sphere is not defined.\n");break;}
            if(s==CYLINDER||s==CONE){r=pos("Radius: ");printf("Base area = %.4f m^2\n",circular_base_area(r));}
            else{l=pos("Length: ");b=pos("Breadth: ");printf("Base area = %.4f m^2\n",cuboid_base_area(l,b));} break;
            case 4:d=pos("Density (kg/m^3): ");printf("Mass = %.4f kg\n",calculate_mass(d));break;
            case 5:printf("Weight = %.4f N\n",calculate_weight());break;
            case 6:do{printf("Elevation (m): ");scanf("%lf",&e);if(e<0)printf("Elevation cannot be negative.\n");}while(e<0);printf("Potential energy = %.4f J\n",potential_energy(e));break;
            case 7:d=pos("Density (kg/m^3): ");printf("Pressure = %.4f Pa\n",calculate_pressure(d));break;
            case 8:fd=pos("Fluid density (kg/m^3): ");printf("Buoyant force = %.4f N\n",buoyant_force(fd));break;
            case 9:sd=pos("Solid density: ");fd=pos("Fluid density: ");res=determine_buoyancy(sd,fd);if(res==SINKS)printf("Solid SINKS.\n");else if(res==FLOATS)printf("Solid FLOATS.\n");else printf("Solid is SUSPENDED.\n");
            case 10:printf("Exiting program.\n");
        }
    }while(ch!=10); return 0;
}

```

Compilation and Execution

```

gcc main.c geometry.c physics.c math_utils.c -o solid_project
./solid_project

```

Sample Verification

Example: choose Cone Surface Area with radius 3 m and height 4 m. Newton-Raphson gives slant height approximately 5 m, so total surface area = $\pi \times 3 \times (3+5) = 75.3984 \text{ m}^2$. For buoyancy behavior, solid density 800 kg/m^3 and fluid density 1000 kg/m^3 produces FLOATS; equal densities produce SUSPENDED; a greater solid density produces SINKS.

Result

The required menu-driven modular C project was designed with separate geometry, physics, square-root utility and header modules. Input validation, SI units, valid/invalid/boundary test cases, Newton-Raphson square root, and all specified geometry and physics operations are included.